Guide du site web

TIC315 — Projet 4, jeu d'échecs en ligne. Comprendre Spring Boot et les parties centrales du code, simplement.

1. L'idée en une minute

Le site fonctionne comme un restaurant :

Au restaurant	Dans le site	Son rôle
Le client	Le navigateur du joueur	Il affiche l'échiquier et envoie les coups
La salle	`ihm`	Elle reçoit les demandes venues du navigateur
Le serveur	`controle`	Il prend la commande et la transmet, sans cuisiner
La cuisine	`metier`	Elle fait le vrai travail : les règles des échecs
Les fournisseurs	`infrastructure`	Stockfish, la base de données, la connexion

Spring Boot, c'est le bâtiment déjà équipé. Il fournit le serveur web et il branche les pièces entre elles. Nous n'avons plus qu'à écrire ce qui est propre à notre jeu.

Règle la plus importante : la cuisine ne connaît pas la salle. Le package `metier` ne contient aucune trace de Spring ni du web. On peut donc le tester seul, avec JUnit, et c'est lui qui est noté.

2. Le vocabulaire

Mot	En clair
Java	Le langage du projet
Spring	Un ensemble d'outils pour écrire des applications Java
Spring Boot	Spring déjà configuré, prêt à démarrer
Maven	L'outil qui télécharge les bibliothèques et construit le programme
`pom.xml`	Le fichier qui dit à Maven de quoi le projet a besoin
JAR (`echecs.jar`)	Le programme final, en un seul fichier, qui se lance avec `java -jar`
Tomcat	Le serveur web, déjà inclus dans le JAR
Annotation (`@...`)	Une étiquette posée sur une classe, que Spring sait lire
Bean	Un objet créé et gardé par Spring
Injection de dépendances	Une classe reçoit les objets dont elle a besoin au lieu de les créer
API	Les adresses que le navigateur appelle pour demander une action
JSON	Le format texte des données échangées : `{"depart": "e2", "arrivee": "e4"}`
HTTP	Le navigateur demande, le serveur répond, et c'est fini
WebSocket	Une ligne qui reste ouverte : le serveur peut parler le premier
Docker	Une boîte qui contient le programme et tout ce qu'il lui faut, pour la mise en ligne

Mot	En clair
PostgreSQL	La base de données où restent les comptes et les parties

Pourquoi HTTP et WebSocket ? Pour jouer un coup, HTTP suffit : le joueur demande et le serveur répond. Mais l'adversaire, lui, n'a rien demandé : seul le WebSocket permet de le prévenir tout de suite.

3. L'injection de dépendances

C'est la seule idée vraiment nouvelle de Spring. Voici les deux façons d'obtenir un objet :

```
// Sans injection : la classe fabrique elle-même ce dont elle a besoin.
private final Partie partie = new Partie();

// Avec injection : elle le reçoit de l'extérieur, et Spring s'en charge.
public ControleurPartie(Partie partie) {
    this.partie = partie;
}
```

Avantage : dans un test, on peut lui donner un faux objet. C'est ainsi qu'on testera le contrôleur avec un faux moteur d'échecs, sans lancer Stockfish.

4. Les parties centrales du code

Ces extraits, sans leurs lignes `import`, viennent d'un petit prototype que nous avons fait tourner (Spring Boot 4.1.2, Java 25). Il a été **simplifié** pour tenir ici : une seule partie, et un coup réduit à deux cases. Le vrai code suivra les diagrammes de conception du dossier.

4.1 Le métier : du Java tout simple

Aucune annotation ici. La partie prévient ses observateurs sans savoir qui ils sont : c'est le patron Observateur.

```
public class Partie {
    private final List<String> coups = new ArrayList<>();
    private final List<ObservateurPartie> observateurs = new CopyOnWriteArrayList<>();

    public void abonner(ObservateurPartie observateur) {
        observateurs.add(observateur);
    }

    public void jouerCoup(String depart, String arrivee) {
        coups.add(depart + arrivee);
        observateurs.forEach(observateur -> observateur.actualiser(this));
    }

    public String dernierCoup() {
        return coups.getLast();
    }
}
```

4.2 Le démarrage : brancher les pièces

`@SpringBootApplication` lance tout. Chaque `@Bean` dit à Spring comment créer un objet : le métier et le contrôleur restent ainsi libres de toute annotation.

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
```

```
    @Bean
    Partie partie() {
        return new Partie();
    }

    @Bean
    ControleurPartie controleurPartie(Partie partie) {
        return new ControleurPartie(partie);
    }
}
```

4.3 Recevoir un coup : l'API

`@RestController` dit que cette classe répond au navigateur. `@PostMapping` donne l'adresse. `@RequestBody` transforme le JSON reçu en objet Java. La classe ne fait que transmettre.

```
@RestController
public class PartieApi {
    record DemandeCoup(String depart, String arrivee) {}

    private final ControleurPartie controleur;

    PartieApi(ControleurPartie controleur) {
        this.controleur = controleur;
    }

    @PostMapping("/api/coups")
    void jouerCoup(@RequestBody DemandeCoup coup) {
        controleur.jouerCoup(coup.depart(), coup.arrivee());
    }
}
```

4.4 Prévenir l'adversaire : la vue

Chaque joueur connecté a sa `VuePartie`. Quand la partie change, la vue envoie la nouvelle au navigateur par le WebSocket.

```
class VuePartie implements ObservateurPartie {
    private final WebSocketSession navigateur;

    VuePartie(WebSocketSession navigateur) {
        this.navigateur = navigateur;
    }

    @Override
    public void actualiser(Partie partie) {
        try {
            navigateur.sendMessage(new TextMessage(partie.dernierCoup()));
        } catch (IOException erreur) {
            throw new UncheckedIOException(erreur);
        }
    }
}
```

4.5 Ouvrir le WebSocket

Quand un navigateur se connecte à `/ws/partie`, on crée sa vue et on l'abonne à la partie.

```
@Configuration
@EnableWebSocket
class ConfigurationWebSocket implements WebSocketConfigurer {
    private final Partie partie;

    ConfigurationWebSocket(Partie partie) {
        this.partie = partie;
    }
```

```
    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registre) {
        registre.addHandler(new TextWebSocketHandler() {
            @Override
            public void afterConnectionEstablished(WebSocketSession navigateur) {
                partie.abonner(new VuePartie(navigateur));
            }
        }, "/ws/partie");
    }
}
```

5. Le chemin d'un coup

Un joueur joue e2-e4. Dans le prototype, ce trajet a fonctionné de bout en bout :

1. Le joueur déplace sa pièce. La page envoie `{"depart": "e2", "arrivee": "e4"}` à `/api/coups`.
2. `PartieApi` reçoit la demande et la transmet à `ControleurPartie`.
3. `ControleurPartie` appelle `Partie::jouerCoup`.
4. `Partie` enregistre le coup. Dans le vrai code, elle vérifie aussi les règles.
5. `Partie` prévient ses observateurs : elle appelle `actualiser` sur chaque `VuePartie`.
6. La `VuePartie` de l'adversaire envoie le coup à son navigateur par le WebSocket.
7. L'échiquier de l'adversaire se met à jour, sans qu'il ait rien demandé.

6. Construire et lancer

```
mvn package                  # construit target/echecs.jar
java -jar target/echecs.jar  # lance le site sur http://localhost:8080
```

Il faut Java 25 et Maven. Le site de démonstration sera ensuite mis en ligne dans un conteneur Docker.

7. Ce qu'il faut retenir

1. Le métier ne dépend de rien : ni de Spring, ni du web.
2. Spring Boot fournit le serveur et branche les objets (injection de dépendances).
3. HTTP sert à demander une action ; le WebSocket sert à être prévenu.
4. Le chemin est toujours le même : navigateur, `ihm`, `controle`, `metier`, puis retour par l'Observateur.